Module 03c — OOP Part 3: Polymorphism and Project

Phase: Advanced Python
Previous: OOP Part 2: Inheritance and Composition
Next: OOP Part 4: Abstract Classes and Interfaces

Series: OOP overview · Part 1 · Part 2 · Part 3 (this file) · Part 4

Lab kit: 15_Lab_Assets/17_bank_account_oop/ — this part is the **CLI** push (class warm-up was Part 1; `SavingsAccount` was Part 2).

Module Outcome

Write one call site that works for many objects, print readable objects with `__str__`, and drive an encapsulated account from a menu.

By the end of this module you will be able to:

- Use override polymorphism (shared parent) and duck typing (shared method name)
- Avoid Java-style overloading (last `def` wins; use defaults or `*args`)
- Override `__str__` so `print(obj)` is useful
- Loop mixed payments or accounts without `if type == ...`
- Finish the BankAccount **CLI** on the existing kit

Part 2 already taught override and hierarchical children. This part **uses** that in a loop. It does not reteach `class` / `self` / inheritance types.

---

1. Why This Concept Matters

Checkout code without polymorphism often looks like this:

```
def checkout(payment_type, amount):
    if payment_type == "upi":
        print(f"Paid ₹{amount} via UPI")
    elif payment_type == "card":
        print(f"Paid ₹{amount} via Card")
    elif payment_type == "cash":
        print(f"Paid ₹{amount} in Cash")
    else:
        print("Unknown payment type")

checkout("upi", 250)
checkout("card", 999)
checkout("wallet", 100)
```

Expected Output:

```
Paid ₹250 via UPI
Paid ₹999 via Card
Unknown payment type
```

Problems:

1. Every new payment mode needs a new `elif`.
2. Easy to miss a branch (silent `"Unknown"`).
3. Type switching and business logic live in one growing function.

Same method name on different objects — **polymorphism** — lets the old checkout loop keep working when you add a class:

```
class UPI:
    def pay(self, amount):
        print(f"Paid ₹{amount} via UPI")

class Card:
    def pay(self, amount):
        print(f"Paid ₹{amount} via Card")

def checkout(method, amount):
    method.pay(amount)

checkout(UPI(), 250)
checkout(Card(), 999)
```

Expected Output:

```
Paid ₹250 via UPI
Paid ₹999 via Card
```

Add `Cash` with `pay()` later. `checkout` does not change.

---

2. Polymorphism — override, duck typing, no overloading

Understand

Polymorphism = “many forms.” The same operation name behaves differently depending on the object. Purpose: write one loop or helper without giant `if type == ...` chains.

Three Python tastes:

Path	What you need	When
Override	Shared parent; child redefines the method	Shared fields / default behaviour (Part 2 Vehicles)
Duck typing	Same method name — no shared parent required	Day-to-day Python: if it has `pay()`, it works
Overloading	Java-style same name, different signatures	**Not supported.** Last `def` wins. Use defaults or `*args`

Python: *if it walks like a duck and quacks like a duck…*

How It Works

Override — same call site, hierarchy from Part 2:

```
class Vehicle:
    def info(self):
        return "Vehicle"

class Car(Vehicle):
    def info(self):
        return "Car"

class Bike(Vehicle):
    def info(self):
        return "Bike"

for v in [Car(), Bike(), Car()]:
    print(v.info())
```

Expected Output:

```
Car
Bike
Car
```

Duck typing — no shared parent:

```
class UPI:
    def pay(self, amount):
        print(f"Paid ₹{amount} via UPI")

class Card:
    def pay(self, amount):
        print(f"Paid ₹{amount} via Card")

class Cash:
    def pay(self, amount):
        print(f"Paid ₹{amount} in Cash")

def checkout(method, amount):
    method.pay(amount)

for method in [UPI(), Card(), Cash()]:
    checkout(method, 100)
```

Expected Output:

```
Paid ₹100 via UPI
Paid ₹100 via Card
Paid ₹100 in Cash
```

No Java-style overload — a class body runs top to bottom; a second `def add` **replaces** the first:

```
class Adder:
    def add(self, *nums):
        return sum(nums)
```

```
print(Adder().add(1, 2))
print(Adder().add(1, 2, 3, 4))
```

Expected Output:

```
3
10
```

What goes wrong with “fake” overloading:

```
class Broken:
    def add(self, a, b):
        return a + b

    def add(self, a, b, c):   # replaces the previous add
        return a + b + c

print(Broken().add(1, 2, 3))
```

Expected Output:

```
6
```

`Broken().add(1, 2)` would be `TypeError: missing c` — Python did not pick by argument count.

Operator overloading (`__add__`, `__len__`) can customise `+` and `len()`. Powerful — use sparingly so code stays readable. Optional extra: Professional Python Deep Dive.

⚠️ WATCH OUT

```
# Wrong — three methods named add; expect Python to pick by arity
def add(self, a, b): ...
def add(self, a, b, c): ...   # this is the only add that remains

# Prefer
def add(self, *nums):
    return sum(nums)
```

Try It

Write `Intern` and `Manager` each with `pay()` (stipend vs basic+bonus). Loop one list and print `name: ₹pay` for each. No shared parent required.

---

3. Readable objects with `__str__`

Understand

`print(account)` should look like `Priya — savings ₹5,000`, not `<Account object at 0x...>`. The hex address is a locker number — useful to the OS, useless to you.

`__str__` controls what you see when you `print(obj)` or convert with `str(obj)`. Keep it short and free of side effects (no network, no files).

`__repr__` is the unambiguous debug form. Implement `__str__` first; dig into `__repr__` in Professional Python Deep Dive if needed.

How It Works

```
class Student:
    def __init__(self, roll, name):
        self.roll = roll
        self.name = name

    def __str__(self):
        return f"Student({self.roll}, {self.name})"

print(Student(101, "Anita"))
print("Label:", str(Student(102, "Vikram")))
```

Expected Output:

```
Student(101, Anita)
Label: Student(102, Vikram)
```

Without `__str__`, `print(Bare("Anita"))` shows `<__main__.Bare object at 0x...>`.

A realistic account:

```
class BankAccount:
    def __init__(self, holder, balance=0):
        self.holder = holder
        self._balance = float(balance)

    def __str__(self):
        return f"{self.holder}'s account — ₹{self._balance:.2f}"

print(BankAccount("Priya", 1000))
print(BankAccount("Ravi", 250.5))
```

Expected Output:

```
Priya's account — ₹1000.00
Ravi's account — ₹250.50
```

`print(obj)` → `obj.__str__()` if defined. Children can override `__str__` and call `super().__str__()` for a typed statement line.

⚠ WATCH OUT

Do not put deposits, file writes, or network calls inside `__str__`. It must be a cheap description — `print` in a loop should be safe.

Try It

Give `Product` a `__str__` of `sku @ ₹price`. Put two products in a list and print them joined with `|`.

Step-by-Step: Build It

One assemble: duck typing checkout **plus** hierarchical accounts printing via `__str__`. These are the two loops the CLI will need — not the menu itself.

```
class UPI:
    def pay(self, amount):
        print(f"UPI paid ₹{amount:.2f}")
        return True

class Card:
    def pay(self, amount):
        if amount > 50000:
            print("Card limit exceeded")
            return False
        print(f"Card paid ₹{amount:.2f}")
        return True

ok = 0
payments = [(UPI(), 250), (Card(), 999), (Card(), 60000)]
for method, amount in payments:
    if method.pay(amount):
        ok += 1
print(f"Successful payments: {ok}/{len(payments)}")

class BankAccount:
    def __init__(self, holder, balance=0):
        self.holder = holder
        self._balance = float(balance)

    def deposit(self, amount):
        if amount > 0:
            self._balance += amount

    def __str__(self):
        return f"{self.holder}: ₹{self._balance:.2f}"

class SavingsAccount(BankAccount):
    def __init__(self, holder, balance=0, rate=0.04):
        super().__init__(holder, balance)
        self.rate = rate

    def apply_interest(self):
        self.deposit(self._balance * self.rate)

    def __str__(self):
        return f"Savings | {super().__str__()} | rate={self.rate:.0%}"

accounts = [
    BankAccount("Anita", 1000),
    SavingsAccount("Priya", 10000, 0.05),
]
accounts[1].apply_interest()
for acct in accounts:
    print(acct)
```

Expected Output:

```
UPI paid ₹250.00
Card paid ₹999.00
Card limit exceeded
Successful payments: 2/3
Anita: ₹1000.00
Savings | Priya: ₹10500.00 | rate=5%
```

Same `print(acct)` call; each object supplies its own string. Same `method.pay(amount)` call; each method supplies its own behaviour.

Practice Exercises

Attempt each problem before reading the model answer.

1. Write three notifier classes (`Sms`, `Email`, `Push`) each with `send(msg)`. Write `notify_all(notifiers, msg)` that loops once.

Hint: Duck typing — no shared parent required.

Model answer:

```
class Sms:
    def send(self, msg):
        print("SMS:", msg)

class Email:
    def send(self, msg):
        print("EMAIL:", msg)

class Push:
    def send(self, msg):
        print("PUSH:", msg)

def notify_all(notifiers, msg):
    for n in notifiers:
        n.send(msg)

notify_all([Sms(), Email(), Push()], "Order shipped")
```

Expected Output:

```
SMS: Order shipped
EMAIL: Order shipped
PUSH: Order shipped
```

2. Add `__str__` to `Employee(emp_id, name)` and print a list of employees joined by commas.

Model answer:

```
class Employee:
    def __init__(self, emp_id, name):
        self.emp_id = emp_id
        self.name = name
```

```
    def __str__(self):
        return f"{self.name}#{self.emp_id}"

team = [Employee(1, "Ravi"), Employee(2, "Meera")]
print(", ".join(str(e) for e in team))
```

Expected Output:

```
Ravi#1, Meera#2
```

3. Show that “overloading” fails: define `show` twice on a class. Call `show(5)` and comment what `show()` with no args would do.

Model answer:

```
class Demo:
    def show(self):
        print("one")

    def show(self, x):   # replaces previous show
        print("two", x)

Demo().show(5)
```

Expected Output:

```
two 5
```

`Demo().show()` would raise `TypeError` (missing `x`). The first `show` is gone.

4. `SmsAlert` and `WhatsAppAlert` each have `notify(patient, msg)`. Write `send_report_ready(channel, patient)` and send `"Lab report ready"` through both.

Model answer:

```
class SmsAlert:
    def notify(self, patient, msg):
        print(f"SMS to {patient}: {msg}")

class WhatsAppAlert:
    def notify(self, patient, msg):
        print(f"WhatsApp to {patient}: {msg}")

def send_report_ready(channel, patient):
    channel.notify(patient, "Lab report ready")

send_report_ready(SmsAlert(), "Anita")
send_report_ready(WhatsAppAlert(), "Ravi")
```

Expected Output:

```
SMS to Anita: Lab report ready
WhatsApp to Ravi: Lab report ready
```

5. Give `Appointment` a `__str__` of `slot | patient → Dr doctor`. Print a two-item queue in a `for` loop.

Model answer:

```
class Appointment:
    def __init__(self, patient, doctor, slot):
        self.patient = patient
        self.doctor = doctor
        self.slot = slot

    def __str__(self):
        return f"{self.slot} | {self.patient} → Dr {self.doctor}"

queue = [
    Appointment("Anita", "Meera Iyer", "10:30"),
    Appointment("Ravi", "Kiran Das", "11:00"),
]
for slip in queue:
    print(slip)
```

Expected Output:

```
10:30 | Anita → Dr Meera Iyer
11:00 | Ravi → Dr Kiran Das
```

6. Override `__str__` on `SavingsAccount` so a mixed list prints a typed line for savings and a plain line for `BankAccount`. Deposit interest on savings first.

Hint: `super().__str__()` plus a `savings` prefix. Same idea as Step-by-Step.

Model answer:

```
class BankAccount:
    def __init__(self, holder, balance=0):
        self.holder = holder
        self._balance = float(balance)

    def __str__(self):
        return f"{self.holder}: ₹{self._balance:.0f}"

class SavingsAccount(BankAccount):
    def __str__(self):
        return f"savings | {super().__str__()}"

for acct in [BankAccount("Anita", 1000), SavingsAccount("Priya", 10000)]:
    print(acct)
```

Expected Output:

```
Anita: ₹1000
savings | Priya: ₹10000
```

Lab

Combine duck typing, one helper function, and `__str__`. Not the BankAccount menu.

Problem

A helpdesk sends the same message through SMS and email. Each channel should print a readable label when you `print` it.

What to Build

`Sms` and `Email` with `send(msg)` and `__str__`. A `notify_all` loop. A short demo.

Requirements

- Both classes implement `send(self, msg)` that prints a prefixed line
- `__str__` returns `"SMS channel"` / `"EMAIL channel"` (or similar)
- `notify_all(channels, msg)` loops once — no `if type == ...`
- Demo: print each channel, then notify `"Ticket T-12 updated"`

Expected Behavior

One function, two behaviours. `print(channel)` is readable, not a memory address.

Sample Run

```
SMS channel
EMAIL channel
SMS: Ticket T-12 updated
EMAIL: Ticket T-12 updated
```

Constraints

No `raise` / `try` / `except`. No files. No BankAccount CLI here — that is the Mini-Project.

Bonus

Add `Push` with the same `send` / `__str__` contract and drop it into the same list.

Mini-Project (Assignment): HDFC Bank BankAccount OOP CLI

Why this matters: Encapsulation, inheritance (optional savings), and `__str__` meet a real menu — the same shape as every ATM / UPI demo.

Lab kit: 15_Lab_Assets/17_bank_account_oop/ — finish `starter/`. Do **not** copy `solution/`.

This is **not** the helpdesk lab. Same kit as Parts 1–2, now with a menu.

Objective

Drive one encapsulated `BankAccount` from a CLI: deposit, withdraw, balance, exit.

Story / Scenario

Campus desk staff type a choice and an amount. They expect a clear message on bad amounts — never a traceback from your validation. (Typing a non-number into `float(input(...))` may still crash this week; Exception Handling covers `try` / `except` next.)

Requirements

Piece	Detail
`BankAccount`	`__balance` (or `_balance` + property); `deposit`, `withdraw`, `get_balance`
Rules	Reject amount ≤ 0; reject withdraw over balance — **print + return**
CLI menu	**1** Deposit · **2** Withdraw · **3** Balance · **4** Exit
Nice to have	`__str__` for a one-line status print

Do not paste a finished class or menu from notes. Use your Part 1–2 starter.

Expected Behavior

- Menu options 1–3 work; 4 exits
- Balance is not casually overwritten from outside
- Overdraft / non-positive amounts print a message and leave balance unchanged

Sample Run

```
Account holder name: Anita
Opening balance: 1000

1) Deposit  2) Withdraw  3) Balance  4) Exit
Choose: 1
Amount: 200

1) Deposit  2) Withdraw  3) Balance  4) Exit
Choose: 2
Amount: 100

1) Deposit  2) Withdraw  3) Balance  4) Exit
Choose: 3
Anita: ₹1100.00
Balance: 1100.0

1) Deposit  2) Withdraw  3) Balance  4) Exit
Choose: 4
Bye
```

Bonus Challenges

- Add `SavingsAccount(BankAccount)` with `apply_interest` **or** a `CurrentAccount` overdraft rule (menu option)
- Log successful deposit/withdraw as a list of tuples `(action, amount, balance)`
- **Later (Exceptions week):** switch validation to `raise ValueError` and catch it in the menu

Done when

- [ ] Lab kit BankAccount ticket passes
- [ ] Menu options 1–3 work; 4 exits

- [ ] Balance is not casually overwritten from outside
- [ ] No `try` / `except` / `raise` required on the core path

Interview Q&A

Q1. Polymorphism in one sentence?
Same operation name, different behaviour depending on the object — so one loop can handle account types or pay methods.

Q2. Override polymorphism vs duck typing?
Override: shared parent, child redefines the method. Duck typing: shared method name is enough; no parent required. Python uses both.

Q3. Why doesn't Python support method overloading like Java?
A class body is executed top to bottom; a second `def add(...)` replaces the first. Use default arguments or `*args` / `**kwargs`.

Q4. `__str__` vs default object print?
Default print shows class name and memory address. Override `__str__` to return a clear, human-readable summary for logs and CLIs.

Q5. Why not `if payment_type == "upi":` forever?
Every new mode needs a new branch; missing branches fail silently. A `pay()` method on each type lets checkout stay one loop.

Q6. Where do `self`, `@property`, and `__balance` fit?
Those are Part 1. Polymorphism does not replace encapsulation — the CLI still hides balance and validates in methods.

Cheat Sheet

```
polymorphism          → same name, different behaviour
override + loop       → for obj in items: obj.action()
duck typing           → if it has pay(), checkout can call it
no overloading        → last def wins; use defaults / *args
__str__               → print(obj) / str(obj) text
default print         → <Class object at 0x...>
super().__str__()     → parent text plus child prefix
```

Validate CLI amounts with print + return this week.

Completion Checklist

```
[ ] I can loop mixed objects with one method name (override or duck typing)
[ ] I know Python does not overload by signature
[ ] I can write __str__ and print a list of objects
[ ] I completed the lab
[ ] I finished the BankAccount CLI on the kit
[ ] I can explain polymorphism vs if-type trees in an interview
```

What's Next?

You can now write one loop over many types and print objects that make sense.

Next, OOP Part 4: Abstract Classes and Interfaces **forces** children to implement the method (`ABC`, abstract methods, interface-style contracts) — then Exception Handling.